Sprint 2 — análisis y plan

Fecha: 2026-10-05 Alcance: columna *Por hacer* del tablero. Estado del análisis: verificado leyendo el código, no de memoria.

> **Estado final: cerrado.** SCRUM-12, 13, 15 y 16 están implementados y verificados. El cierre, con los números reales de tests y los comandos para reproducirlos, está al final: Estado final. El resto del documento se conserva como está, porque las decisiones que se tomaron ahí siguen siendo la razón de por qué el código es como es.

Alcance del sprint

Ticket	Título	Área
SCRUM-12	Paneles Admin, Pacientes, Medicos y Recepcionistas	Back
SCRUM-13	Paneles Admin, Pacientes, Medicos y Recepcionistas	Front
SCRUM-14	Test Register y funcionalidades	Testing
SCRUM-15	Comenzar el sistema de turnos (0/1 subtareas)	Back
SCRUM-16	Gestión de especialidades	Back

Dónde vive cada cosa

Dos repositorios separados, ambos en GitHub y con `main` / `develop` sincronizadas:

- **Back**: `C:\Users\PC\Desktop\proyectos\clinica_back` → `matiiiazc/clinica-back`
- **Front**: `C:\Users\PC\Desktop\proyectos\clinica_front` → `matiiiazc/clinica-front`

> Existe además `C:\Users\PC\Documents\Default Project` con un backend Express + Prisma conectado a Neon. **No forma parte de este sprint.** Quedó en cero (sin servidor, sin auth) y se ignora hasta que se decida su futuro.

Punto de partida real

`apps/accounts` — terminado ✅

Es lo único del dominio que está completo.

- 8 rutas: `register`, `verify-email`, `resend-code`, `login`, `refresh`, `logout`, `me`, `change-password` (más un `HealthView`).
- 7 serializers: `User`, `Register`, `VerifyEmail`, `ResendCode`, `Login`, `ChangePassword`, `UpdateProfile`.
- Throttling y un backend de autenticación propio en `backends.py`.
- **52 tests en verde** (49 en `tests.py`, 3 en `test_http.py`).

`apps/clinica` — modelos listos, API inexistente ⚠️

Este es el hallazgo central del sprint.

Archivo	Estado
`models.py`	**306 líneas, 6 modelos completos** ✅
`migrations/0001_initial.py`	Creada y **aplicada** ✅ (sin commitear)
`views.py`	**3 líneas**, solo el comentario por defecto ❌
`urls.py`	`urlpatterns: list[path] = []` ❌
`serializers.py`	**No existe** ❌
`tests.py`	Stub de 63 bytes ❌
`admin.py`	Stub, ningún modelo registrado ❌

Los 6 modelos están bien hechos: `Especialidad`, `PerfilPaciente`, `PerfilMedico`, `Agenda`, `Bloqueo`, `Turno`, con la `ExclusionConstraint` de PostgreSQL que prohíbe el solapamiento de turnos en la base.

O sea: hay todo el modelo de datos y ninguna capa HTTP.

Front — auth listo, paneles inexistentes ⚠️

Pieza	Estado
`Register.tsx`, `Login.tsx`, `VerifyEmail.tsx`, `ChangePassword.tsx`	Listas ✅
`AuthProvider`, `useAuth`, `RutaPrivada`	Listas ✅
`lib/api.ts`, `session.ts`, `types.ts`, `ui.tsx`	Listas ✅
`App.tsx` con 6 rutas (`/`, `/login`, `/registro`, `/verificar`, `/cambiar-contrasena`, `/app`)	✅

Pieza	Estado
`Main.tsx`	**Placeholder.** Es donde van los 4 paneles ❌
Ruteo por rol	**No existe.** No se lee `usuario.rol` para decidir el panel ❌

No hay protección por rol en el front: cualquier usuario autenticado cae en `/app`, que es un placeholder.

Gap por ticket

Los porcentajes son una estimación de esfuerzo restante, no una medición.

SCRUM-16 — Gestión de especialidades · ~30% hecho

Lo único que falta es la capa HTTP, porque el modelo ya está:

- `GET/POST /api/clinica/especialidades`
- `GET/PATCH/DELETE /api/clinica/especialidades/:id`
- Registrar los 6 modelos en `admin.py`
- Filtros: solo `activa=True` al agendar; baja lógica en vez de borrar (hay turnos históricos con `PROTECT`)

Es el ticket más chico y el mejor para arrancar: desbloquea SCRUM-15.

SCRUM-15 — Sistema de turnos · ~25% hecho

El modelo está completo y con la constraint que garantiza que no se pisen. Falta:

- `serializers.py` completo: alta, reasignación, cambio de estado, filtro por fecha y por médico
- Endpoints de agenda del día, disponibilidad y turnos
- Calcular slots libres a partir de `Agenda` + `Bloqueo` + turnos existentes (`TURNO_SLOT_MINUTOS` ya está en settings)
- Respetar `Turno.TRANSICIONES` y `ESTADOS_OCUPAN` al cambiar el estado
- Manejar el error de la `ExclusionConstraint` como 409, no 500

> `ESTADOS_OCUPAN = [pendiente, confirmado, atendido]` — un turno cancelado libera el horario. Es intencional, ya está en el modelo.

SCRUM-12 — Paneles back · ~10% hecho

Cero endpoints de dominio. Todo por hacer. Además hay que definir la matriz de permisos por rol, porque las clases ya existen pero **no están aplicadas en ningún lado**.

SCRUM-13 — Paneles front · ~10% hecho

La base está (sesión, `RutaPrivada`, cliente API, UI). Falta:

- Ruteo por rol: leer `usuario.rol` y mandar a `/admin`, `/medico`, `/paciente` o `/recepcion`
- 4 paneles con sus vistas y formularios
- Estados de carga y error (base ya está en `ui.tsx`)
- `npm test` — **no hay tests en el front, la cobertura es manual**

SCRUM-14 — Test de register · ~70% hecho

`register` ya tiene 49 tests de unidad y 3 de HTTP, y el flujo se validó de punta a punta. Falta:

- **Ajuste al frontend.** El registro necesita **SMTP configurado.** Hoy `SMTP_HOST`, `SMTP_USER` y `SMTP_PASSWORD` están vacíos y el backend imprime el código en consola (`dev_code`). Sin eso no se puede probar el envío real. **Es un bloqueo externo, hay que elegir proveedor.**
- Casos que hoy no están cubiertos: email duplicado, código expirado, código incorrecto, rate limit excedido, password TooShort frente a la política de `settings.py`
- E2E del front contra el back real

Huecos transversales

1. **Los permisos existen pero no aplican.** `IsStaffUser`, `IsAdminUser`, `IsPatient` e `IsSelfOrStaff` están escritas en `apps/accounts/permissions.py` y **cero views las importan**, porque no hay views. La matriz de quién ve qué hay que decidirla antes de escribir SCRUM-12, no después.
2. **No hay tests en `apps/clinica`.** `tests.py` es un stub. Cada ticket de este sprint debería traer sus tests.
3. **No hay tests en el front.** `npm test` no existe. SCRUM-13 y SCRUM-14 lo necesitan o quedan sin cobertura automática.
4. **Cambios sin commitear en `main`.** El backend tiene modificados `apps/clinica/models.py` y `clinica_back/settings.py`, y sin trackear `apps/clinica/migrations/0001_initial.py`, `Diagrama UML.png` y `estrucura`. Los modelos ya están aplicados en la base local: **si se pierde ese archivo sin commitear, se pierde la migración aplicada.**
5. **La base local tiene los 6 modelos ya migrados** pero **cero datos de dominio**. No hay especialidades ni profesionales cargados, así que las pruebas de turnos necesitan factories o seeds.

Orden sugerido

1. **Commitear lo que ya está** (los modelos y la migración están aplicados y sin commitear). Arranca el sprint con el repo en un estado conocido.
2. **Elegir proveedor SMTP** — desbloquea SCRUM-14 y las notificaciones de turno.
3. **SCRUM-16 especialidades** — el más chico, y SCRUM-15 depende de él.
4. **SCRUM-15 turnos** — la constraint ya garantiza que no haya doble booking.
5. **SCRUM-12 back** — con la matriz de permisos decidida antes de escribir.
6. **SCRUM-13 front** — se puede empezar el ruteo por rol en paralelo.
7. **SCRUM-14** — cierra cuando SMTP esté listo y los paneles existan.

Decisiones pendientes

#	Pregunta	Bloquea
1	¿Qué proveedor SMTP?	SCRUM-14, avisos de turno
2	~~¿Quién ve qué? matriz por rol~~	Resuelto abajo, con 3 huecos
3	¿Se ponen datos de prueba con factories o con un seed?	SCRUM-15, SCRUM-16
4	¿Neón reemplaza la base local de Django, o son dos bases?	Nada aquí, pero define dónde se prueba
5	¿Los paneles se autentican con cookie o con token?	SCRUM-13

Matriz de visibilidad por rol — DECIDIDO

Definido por el usuario el 2026-10-05, rol por rol. Es la fuente de verdad; el modelo todavía **no** lo soporta entero.

Admin

Ve todo. Sin restricciones. Admin es el único que ve los cuatro paneles.

Médico

- Ver **los turnos que tiene en el día, la semana y el mes**.
- **No agenda.** Los turnos se los asignan recepcionistas.
- Puede cambiar el estado de sus turnos a: *libre*, *atendiendo*, *inactivo*.

Paciente

- **Pedir turno** y **elegir especialidad**.
- **Salvar dudas.** (término sin definir, ver abajo)
- **Ver su historial médico.**
- **Modificar su ficha médica**: afecciones y cosas a tener en cuenta en la parte de salud.
- **La ficha médica se pide desde el register**, no después.

Recepción

- Ver **los médicos disponibles**.
- **Asignar turnos.**
- **Ver la cola de espera.**
- Ver los turnos disponibles.

Qué falta en el modelo para cumplir esto

Lo que pide el rol	Estado actual
Ficha médica del paciente (afecciones, etc.)	**No existe el modelo**
Pedir turno elegindo especialidad	No hay API
Cola de espera	**No existe el modelo**
Ver turnos por día / semana / mes	No hay API
Estados *libre* / *atendiendo* / *inactivo*	**Chocan con** `Turno.Estado`
Historial médico del paciente	No hay API

Esto ya excede el alcance original del sprint. `PerfilPaciente` tiene documento, fecha de nacimiento, dirección y obra social, pero **nada** de salud ni relación con un médico, y no hay ningún concepto de espera.

Matriz de visibilidad por rol — DEFINICIÓN PREVIA

Definido por el usuario el 2026-10-05:

- **Admin**: ve todo, sin restricciones.
- **Médico**: ve los pacientes que tiene asignados.
- **Recepción**: ve los médicos.

Tres huecos que hay que cerrar antes de escribir los permisos

1. "Los pacientes que tengo" no existe todavía como relación.

`PerfilPaciente` (`apps/clinica/models.py:66`) tiene `documento`, `fecha_nacimiento`, `direccion` y `obra_social`. **No tiene un campo que lo vincule a un médico.** Hay dos formas de resolverlo y son distintas:

- **Derivada** — "mis pacientes" = los que tienen al menos un turno conmigo. No requiere cambios de schema. El problema: un paciente que nunca agendó no aparece, y en la primera consulta del panel la agenda del médico viene vacía.
- **Asignación explícita** — un `ForeignKey` de `PerfilPaciente` a `PerfilMedico`. Es lo que hace que un paciente se vea desde el primer día, y permite "mi paciente propio". Requiere migración nueva.

Recomendación: asignación explícita, con la derivada como complemento. La agenda de un médico recién cargado no debería verse vacía.

2. No está definido qué ve un paciente.

Lo que no se dice: un paciente no debería ver la agenda de otros ni la de otros profesionales. Supuesto por defecto: **solo sus propios datos y sus propios turnos.** Confirmar.

3. No está definido qué puede hacer recepción además de ver médicos.

Solo se dijo que recepción ve los médicos. Pero recepción es quien agenda, así que por defecto asumiría:

- Ve también los pacientes (necesita registrarlos para poder agendar)
- **Escriben** turnos: crear, reasignar, cancelar
- Cambian el estado de un turno
- Manage especialidades (SCRUM-16)

Si recepción solo va a *mirar* médicos y nada más, hay que corregirlo.

Lo que falta definir por rol

Rol	Lectura	Escritura	Sin definir
Admin	Todo	Todo	—
Médico	Sus pacientes, su agenda, sus turnos	Cambiar estado de sus turnos	¿Puede ver historial de otro médico?

Rol	Lectura	Escritura	Sin definir
Recepción	Médicos, pacientes, especialidades, turnos	Turnos y especialidades	¿Puede dar de alta profesionales?
Paciente	Sus datos, sus turnos	Pedir turno, cancelar el suyo	¿Puede ver el motivo de consulta?

Las 3 preguntas — RESUELTAS el 2026-10-05

1. Estados: se conservan los del turno y se agregan los del médico

Los 5 estados de `Turno.Estado` quedan como estaban. Los estados *libre / atendiendo / inactivo* son un campo nuevo `PerfilMedico.estado`, que reemplaza al booleano `activo` que existía antes. No se convirtieron en estados de turno porque *libre* no describe un turno reservado: describe al profesional.

`PerfilMedico.disponible` es True solo con `libre`. `atendiendo` está ocupado ahora pero el día sigue; `inactivo` no recibe turnos y conserva su historial.

2. "Salvar dudas" es preguntar a recepción

Modelo `ConsultaRecepcion`. El paciente escribe un mensaje, recepción lo responde. Estados `abierta` / `respondida`. Guarda quién respondió y cuándo.

`paciente` apunta a `User` y no a `PerfilPaciente`, para poder preguntar algo sin tener que tener ficha clínica cargada.

3. Cola de espera — diseñado en esta ronda

Modelo `ColaEspera`. Decisiones tomadas:

- **Se encola contra una especialidad, no contra un médico.** Quien espera cardiología no eligió cardiólogo, y atarlo a uno deja la cola vacía en cuanto ese médico no tenga hueco. El campo `medico` existe pero es opcional, para cuando el paciente sí pide uno en particular.
- `prioridad` **es un entero** (`NORMAL=0`, `URGENTE=10`), no un texto. El orden es urgente primero; con un texto habría que resolverlo con `Case/When` o confiando en que "urgente" ordene antes que "normal" alfabéticamente.
- **Salen por orden de llegada dentro de la misma prioridad.**
- **Estados**: `esperando` / `agendado` / `descartado`. Al agendar se enlaza el turno creado con `ColaEspera.turno`.
- **Una sola entrada viva por paciente y especialidad.** La constraint es parcial y solo mira las `esperando`, así que un paciente que ya fue agendado puede volver a encolarse después.
- **Ventana de fechas opcional** (`desde` / `hasta`) para quien no puede atender cualquier día. Sin fechas, cubre todo.

Implementado en esta ronda

- `PerfilMedico.estado` reemplaza a `PerfilMedico.activo`, con `disponible`.
- `ConsultaRecepcion` nuevo.
- `ColaEspera` nuevo, con sus constraints e índices.
- Migración `clinica/0002_estado_medico_consulta_y_espera`, aplicada.
- `scripts/verificar_modelos.py`: 19 comprobaciones contra la base real, todas en verde. Incluye verificar que la `ExclusionConstraint` de turnos sobrevivió a la migración.

- Los 52 tests de `accounts` siguen pasando: `python manage.py test apps`.

> `python manage.py test` **sin argumentos no encuentra nada**. Hay que pasar `apps`, que es lo que hace la CI. No es un bug, pero sorprende.

Sigue pendiente de definir

La ficha médica del paciente. El paciente tiene que poder ver y modificar su historial y su ficha de salud (afecciones, etc.), y se pide **desde el register**. No existe el modelo.

Hace falta decidir si lo que carga el paciente queda marcado como *no verificado* hasta que un profesional lo confirme. La pregunta se hizo y quedó sin responder: si el paciente puede cambiar sus alergias en cualquier momento, un médico puede atender con un dato que nadie revisó.

Estado final del sprint 2

Fecha de cierre: 2026-10-05. Todo lo de arriba quedó implementado.

Lo que hay ahora

`apps/clinica` **— API completa.** 9 modelos, 23 endpoints bajo `/api/`, serializers con validación de negocio, permisos por rol, servicios de disponibilidad/agenda/bloqueos. En el admin están registrados los 6 que interesan operar a mano (los otros 3 son técnicos o se manejan desde un flujo).

`apps/accounts` **— 1 endpoint más.** `GET /api/auth/usuarios` (solo admin, filtra por `rol` y busca por `q`), que es lo que permite ver las cuentas desde el panel de administración.

Front — 4 paneles. `Main.tsx` lee `usuario.rol` y despacha a `PanelAdmin`, `PanelRecepcion`, `PanelMedico` o `PanelPaciente`. Cada uno con sus secciones sobre los componentes de `components/ui.tsx`.

Números

Qué	Resultado	Cómo se comprueba
Tests del backend	**159 OK**	`python manage.py test apps --noinput`
Typecheck del front	Sin errores	`npm run typecheck`
Lint del front	0 errores, 0 warnings	`npm run lint`
Build del front	OK	`npm run build`
Paneles contra el back real	**39/39 OK**	`npm run verificar:paneles`

`npm run verificar:paneles` es lo más útil de los dos repositorios juntos: no mockea nada, entra por la API con los cuatro roles y camina lo que hace cada uno. Sabe detectar un endpoint caído o un permiso mal puesto.

Datos de demostración

Los paneles se veían vacíos sin contenido. `datos_demo` lo resuelve:

```
python manage.py datos_demo          # muestra el plan, no escribe
python manage.py datos_demo --si     # escribe
```

Crea 4 especialidades, 4 profesionales con agenda, 4 pacientes con ficha, turnos de esta semana y de la que viene, 2 consultas y 2 entradas de cola. Todo con la contraseña `Clinica2026!Demo`. Es idempotente: correrlo dos veces no duplica.

Bugs que aparecieron al verificar de punta a punta

Estos no los veían los tests, y conviene recordarlos:

1. `paciente_id` **como entero en tres serializers.** `User.id` es UUID, así que `TurnoSerializer`, `ConsultaSerializer` y `ColaEsperaSerializer` mandaban un id que no matcheaba ninguna URL. Ahora usan `serializers.UUIDField`. Hay un test de round-trip que sube por `/api/pacientes/<uuid>/ficha`.
2. `get_permissions` **sobreescrito devolviendo permisos sin instanciar.** En `MedicoAgendaView` estaba `IsStaffUser() | IsAdminUser()`, que tira `TypeError` → 500 en cada llamada al GET de la agenda. El POST pasaba porque va por otra rama del método, así que ningún test lo tocaba. Ahora hay siete tests sobre esa vista.
3. `IsStaffUser | IsAdminUser` **sobra en varios** `permission_classes`**.** `IsStaffUser` ya incluye a los administradores, así que la combinación es redundante. No es un bug, queda como nota.

Lo que sigue pendiente

- **SMTP.** El registro sigue mostrando `dev_code` en consola. Es el único bloqueo externo que queda de SCRUM-14.
- **Sin tests automatizados en el front.** La verificación es manual o con el script de `verificar:paneles`.
- **Neón.** Sigue sin decidirse si reemplaza la base local, y la credencial que se compartió en el chat hay que rotarla antes de publicar el proyecto.
- `README.md` **borrado.** No se recrea salvo que se pida.